

suffix arrays [?]. However, as we are using a tolerance-based matching of numeric values, techniques such as suffix trees and suffix arrays do not work in our case. Implementing the LZ entropy rate estimator adapted to numeric data in a naive way has a time complexity of $O(n^3)$. We develop an algorithm based on dynamic programming that brings down the run time to $O(n^2)$.

In summary, we make the following contributions:

- to do

II. RELATED WORK

The related work section is divided into two parts. First, we have a look at entropy-based approaches for computing or estimating the predictability of time series. In the second part, we review techniques for implementing approaches to estimate entropy rates.

Entropy is a well-known concept applied to estimating predictability [?]. Song et al. propose an approach based on the entropy rate of a (human mobility) time series to compute an upper bound of predictability [?]. Although criticized by Smith et al. [?] and Xu et al. [?] for using a coarse spatial granularity resulting in a loose upper bound, in principle the approach by Song et al. works. However, it can only be applied to symbolic time series data. This is one of the reasons why a relatively coarse granularity was used: mapping spatial areas to symbols loses the spatial relationships between the different areas. Generalizing an entropy-based approach from a discrete domain to a continuous domain is far from trivial. As we will show later, the naive discretization applied by Smith et al. causes problems in terms of the accuracy. The straightforward approach of replacing the sum in the formula for entropy with an integral (as originally suggested by Shannon [?]) leads to the so-called *differential entropy* [?], [?], which has several issues, such as easily becoming infinite for continuous probability distributions [?] and not being invariant under coordinate transformations [?].

Since the seminal work of Shannon [?], [?], many different notions of entropy, such as approximate entropy [?], fuzzy entropy [?], sample entropy [?], and permutation entropy [?], have been proposed. An advantage of these entropy types is that they can be applied to continuous domains. For instance, Chen et al. [?] have estimated the predictability of numeric time series with fuzzy entropy, approximate entropy, and sample entropy, while Garland et al. [?] have done so with permutation entropy. Garland et al. demonstrate empirically that their weighted permutation entropy (WPE) is correlated to the mean absolute scaled error (MASE). The method is limited to correlation, though, as WPE and MASE are two completely different measures and there is no direct connection between the two. This lack of being directly comparable with entropy is a general issue with measures for forecast accuracy, such as MASE (even though MASE is widely considered to be the preferred measure [?]). We are not aware of any method to compute an upper bound of the best achievable forecast accuracy for a given numeric time series. Summarizing the entropy-based state-of-the-art methods, we can say that on the one hand, there are techniques that compute an upper

bound for the intrinsic predictability of a time series but only work for discrete time series [?]. On the other hand, there are approaches that can be applied to numeric time series, but which only establish a correlation between forecasting accuracy and entropy and do not provide an upper bound for the intrinsic predictability. With our method, we can compute an upper bound for the intrinsic predictability for numeric time series.

We now turn to the second part of the related work section, implementing the entropy rate computation of a time series. First of all, we point out that calculating the exact value of the entropy rate is usually not possible, as the exact probability distribution of the underlying generating process is not known (we assume that we are only given the observed values). Even if we had access to the concrete probabilities, it would still be infeasible to compute the exact entropy rate, since this has high runtime costs. Consequently, the entropy rate of a time series is usually estimated rather than computed exactly. For an overview and a discussion, see [?], in which Feutrill distinguishes between parametric estimators, which assume that the data generation follows a certain model, such as a Gaussian process [?] or a Markov process [?], and non-parametric ones, which do not make this assumption. We disregard parametric estimators, as we do not make any assumptions about the underlying model. For non-parametric estimators, such as the Lempel-Ziv-based algorithm [?], the close relationship between entropy and data compression algorithms is exploited. Intuitively, the entropy rate measures how many bits of novel information are introduced by each new sample emitted from a source. When using a compression algorithm on these samples, the compression rate will be higher if less novel information is encountered and vice versa. The Lempel-Ziv (LZ) algorithm is of particular interest, as it is a *universal source code* for ergodic, stationary sources. A code for a source X (e.g. a random variable) is a mapping from the domain of X to a set of symbols. For a universal source code, the limit of the expected code length (for the number of samples going to infinity) reaches the entropy rate and the mapping is independent of the particular source [?]. There are different variants of LZ, depending on how the data is parsed. In the 1978 paper by Lempel and Ziv [?], the parsing searches for the shortest prefix of the part of the time series not parsed yet that is not in the dictionary. Cover and Thomas call this a tree-structured Lempel-Ziv algorithm, as it can be viewed as building a (tree-structured) dictionary [?]. On the other hand, Wyner and Ziv look for the shortest prefix of the part of the time series not parsed yet that has not been encountered anywhere in the parsed subsequence [?]. In order to obtain more stable and accurate results, Kontoyiannis et al. use this variant, compute the lengths of all words for every starting position within a time series, and average over all positions [?]. We utilize the results of Kontoyiannis and provide more details on this variant later on. Nonetheless, all of the estimators discussed above are only suitable for symbolic data, i.e., we have to develop a variant that is able to handle numeric time series. In this context, we need algorithms that can efficiently find longest matching subsequences in numeric time series. While there are a significant number of

algorithms for symbolic sequences (e.g. [?], [?], [?], [?], [?], just to name a few), suitable approaches for tolerance-based matching are few and far between. The closest we could find are interval-based methods [?], [?]. We discuss this in more detail in Section VII.

III. PRELIMINARIES

Before we begin describing our approach, we introduce some basic definitions of time series and entropy-related concepts.

Definition 1: A **time series** $\mathbf{T}$ is a time-ordered sequence $T = (x_1, x_2, \dots, x_n)$, where x_t is the value at time point t and n is the length of time series T .

Definition 2: A **time series data generating process** is defined via a stochastic process X , which consists of a sequence of random variables $X_1, X_2, \dots, X_n$. The index refers to the point in time, i.e., X_t is the random variable for time point t .

For the moment, we make the following assumption on the time series T and the stochastic process X :

- Discrete, equi-spaced intervals of time: for $t \in \{1, 2, \dots, n\}$ the delta between any consecutive observations x_t and x_{t+1} is constant; so, X is a discrete time stochastic process.
- Boundedness of values in T : $x_t \in [\min(T), \max(T)]$.
- Stationarity of X : the statistical properties of T do not change over time. More formally, the probabilistic behavior of a sequence $(x_i, x_{i+1}, \dots, x_{i+k})$ is the same as that of a shifted sequence $(x_{i+h}, x_{i+h+1}, \dots, x_{i+h+k})$ [?].
- Ergodicity of X : the average over ensembles and time are the same (ensembles are essentially a collection of possible states). For our case, this means that the average over potential subsequences (for a time period) is the same as the average over a single very long time period [?].

As we cover numeric time series, one option would be to use continuous random variables to define the stochastic process describing a time series. As already discussed in Section II, applying differential entropy, which would be needed for continuous random variables, has several issues. Instead, we look at the entropy rate of a random (numeric) process at a certain resolution, treating values within a distance of ϵ as indistinguishable. We provide details on how this works in V-B.

Definition 3: **Entropy** measures the information content, quantifying the uncertainty of a random variable. For random variable Y over domain ϕ

$$H(Y) = - \sum_{y \in \phi} p(y) \log_2(p(y)) \quad (1)$$

$p(y)$ is the probability that Y takes value $y \in \phi$. Entropy is usually measured in bits, thus we use the logarithm to the base 2.

Entropy is always greater or equal to 0 (assuming that $0 \log_2(0) = 0$) and is maximized for a uniformly distributed random variable. Additionally, entropy always refers to a single random variable. As a time series is defined by a

stochastic process made up of a sequence of random variables $X_1, X_2, \dots, X_n$, we need to modify how to measure the uncertainty of the process. Instead of using entropy, we use the *entropy rate*, which intuitively measures the average uncertainty per symbol in the generated time series.

Definition 4: The **entropy rate** $\mathcal{H}$ of a stochastic process X consisting of the random variables $X_1, X_2, \dots, X_n$ is computed with the help of conditional entropy and the limit as n tends to infinity:

$$\mathcal{H}(X) = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{t=1}^n H(X_t | X_{t-1} \dots X_1) \quad (2)$$

We should mention that the limit may not exist if the generating process X is non-stationary or non-ergodic. Moreover, for $t=1$ the above equation is reduced to $H(X_1)$ [?].

Definition 5: The **conditional entropy** measures the amount of information provided by the outcome of a random variable Y given the value of another random variable Z .

$$\begin{aligned} H(Y|Z) &= \sum_{z \in \zeta} p(z) H(Y|Z=z) \\ &= - \sum_{z \in \zeta} \sum_{y \in \phi} p(y, z) \log_2 p(y|z) \end{aligned} \quad (3)$$

In our proofs, we use the following theorems on conditional entropy. The proofs of these theorems can be found in [?], along with more details about information theory and entropy.

Theorem 1: Chain rule for joint and conditional entropies:

$$H(X, Y) = H(X) + H(Y|X) \quad (4)$$

Corollary 1: A corollary of this rule is

$$H(X, Y|Z) = H(X|Z) + H(Y|X, Z) \quad (5)$$

Theorem 2: Conditionality reduces entropy:

$$H(X|Y) \leq H(X) \quad (6)$$

IV. MAJOR CHALLENGES

When devising an algorithm to compute an upper bound for the predictability of numeric time series, we faced two major challenges. First, how do we handle numeric data in the first place? We quickly found out that applying discretization methods in a straightforward way does not work. Second, how do we implement efficient LZ-based entropy rate estimators? In the following, we briefly sketch the implications of these challenges.

A. Discretizing Numeric Data

While Song et al. managed to connect predictability directly to entropy, they sidestepped the whole issue of symbolic versus numeric data by assuming that the data is already discretized [?]. Smith et al. continued where Song et al. had left off and explicitly discretized the geolocation data by placing a grid on top of maps [?]. The numeric value domain was essentially divided into bins and all values contained in a bin were assigned the same symbol.

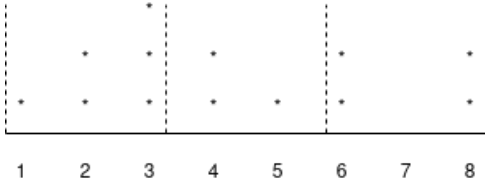


Fig. 1: Binning time series values

Although binning makes it easy to determine whether a prediction is considered acceptable or not (two values sharing the same symbol match), there are still problems regarding proximity. For example, given a value of 6 and two predictions 5 and 8, intuitively 5 is a better prediction, since it is closer. However, binning rarely captures the proximity correctly. In Figure 1, we have a bin width of 2.4, but for this bin configuration, 8 would be considered a matching prediction, while 5 would not be. Assuming that two values are considered close enough when they are within distance 1.2 of each other, we have a total of 78 potentially matching pairs in Figure 1, 25 of which are within range of each other. However, using the binning depicted in the figure with dashed lines, we have 24 matching pairs, 7 of which are false positives, while missing 8 false negatives.

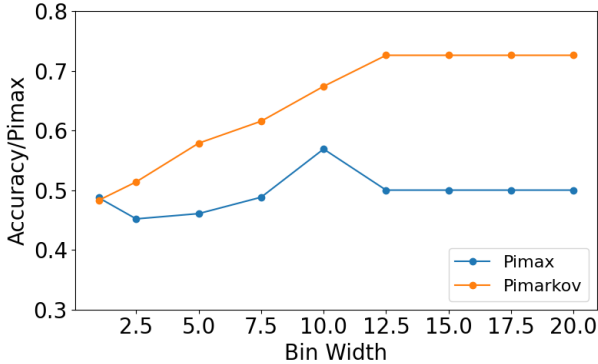


Fig. 2: Proximity Issues due to binning

This is not only counterintuitive, but also leads to scenarios in which Π^{max} ceases to be an upper bound. We generated a synthetic series of integers using a first order Markov model with a total of 50 unique values and trained a model on this data. series, discretizing the values using different bin widths. Figure 2 shows the results. Not only does the model (labeled Pimarkov) have predictability better than the upper bound (labeled Pimax), the upper bound does not grow monotonically with the bin width (we would expect higher predictability for wider bins, due to a decrease in entropy), but it fluctuates depending on where the bounds of the bins are located. This observation highlights the failure of numeric value discretization as a method for computing the predictability upper bound of numeric sequences.

B. Efficient Numerical Entropy Rate Estimators

While running, the LZ algorithm has to find longest matching substrings over and over again, so in his efficient

implementation of entropy estimation, Sadakane used suffix trees to speed up exactly this part of the algorithm [?]. The correctness of suffix trees and suffix arrays relies on the prefix equality of strings or sequences being well-defined. However, when generalizing entropy-based predictability to numeric data, we give up the transitivity of equality when comparing two numbers x and y using a tolerance threshold, i.e., we consider x and y equal, if $|x - y| \leq \epsilon$ (more details on this in Section V). We can easily choose three numbers, x , y , and z , such that $|x - y| \leq \epsilon$, $|y - z| \leq \epsilon$, and $|x - z| > \epsilon$, e.g. $x = 1$, $y = 2$, $z = 3$, and $\epsilon = 1$.

The loss of transitivity means that the concept of a common prefix in a string or sequence is not well-defined anymore, which has critical consequences for constructing suffix trees or arrays. Let us take the sequence xyz , with values from the example above. Then we have three different suffixes: xyz , yz , and z . Following our definition of equality, yz is a prefix of xyz and z is a prefix of yz . However this is a contradiction to x not being equal to z , as it implicitly treats x and z as being equal (because z is a prefix of yz , which is a prefix of xyz). This makes it impossible to construct a consistent suffix tree, as x and y as well as y and z need to be on the same branch, but x and z should not be. As a consequence, this makes it much harder to implement an efficient LZ entropy rate estimator.

V. COMPUTING INTRINSIC PREDICTABILITY

Our goal is to develop a method allowing us to compute an upper bound for the intrinsic predictability of a numeric time series directly from its entropy rate. In order for this to work, we need to consider the proximity of predicted values to actually measured values rather than applying discretization in a straightforward manner.

A. Defining Predictability and Tolerance Threshold Matching

We assume a forecast horizon of one, i.e., given the measured values so far, we predict the next value. After predicting a value, we get access to the next true value, which can then be used for the following prediction. We first define predictability formally and come back to its implementation later.

We start by defining what it means that a predicted value matches the actually measured one.

Definition 6: Given the stochastic process $X = (X_1, X_2, \dots, X_n)$ generating the time series T , let $\hat{X} = (\hat{X}_1, \hat{X}_2, \dots, \hat{X}_n)$ be the process generating the predictions. Given a predefined threshold ϵ , we say that there is a (prediction) **error**, if the correct value x_t does not fall within the ϵ -range $B(\hat{x}_t) = [\hat{x}_t - \epsilon, \hat{x}_t + \epsilon]$, i.e., $|x_t - \hat{x}_t| > \epsilon$. We define the error with the help of the discrete random variable E_t at timestamp t :

$$E_t = \begin{cases} 1 & x_t \notin B(\hat{x}_t) \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

In the following, when we state that $X_t \approx_\epsilon \hat{X}_t$, we mean that $|x_t - \hat{x}_t| \leq \epsilon$ (thus, $X_t \not\approx_\epsilon \hat{X}_t$ means $|x_t - \hat{x}_t| > \epsilon$). Applying $\approx_\epsilon$ to a sequence of values means that $X_i, X_{i+1}, \dots, X_{i+l} \approx_\epsilon \hat{X}_i, \hat{X}_{i+1}, \dots, \hat{X}_{i+l}$ is true, iff $\bigwedge_{j=0}^l (X_{i+j} \approx_\epsilon \hat{X}_{i+j})$ is true.

Definition 6 is used, for instance, when applying information theory concepts to continuous state spaces [?], [?]. There is a subtle but crucial difference compared to binning. Using a tolerance threshold for the distance between values avoids misclassifications: all pairs within distance ϵ will be matched, while those that are further apart will not. We use the definition of a prediction error to define the predictability of a given model (described by its stochastic process $\hat{X}$) for a particular time series.

Definition 7: We distinguish different levels of **predictability**.

- The predictability $\Pi_t^{\hat{X}}$ of $\hat{X}$ for a single step t is equal to

$$\Pi_t^{\hat{X}} = p(E_t = 0) = 1 - p(E_t = 1) \quad (8)$$

- Calculating the average predictability for the process up to timestamp n boils down to

$$\Pi^{\hat{X}}(n) = \frac{1}{n} \sum_{t=1}^n \Pi_t^{\hat{X}} \quad (9)$$

- The overall predictability of $\hat{X}$ is the limit of Equation (9) with n tending to infinity:

$$\Pi^{\hat{X}} = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{t=1}^n \Pi_t^{\hat{X}} \quad (10)$$

Applying Equation (7) assumes that we know the threshold ϵ . This raises the question of where we get the value from. Morse and Patel give an answer to this question when using a threshold ϵ in the context of determining the similarity of two time series [?]. In this case, x_t and $\hat{x}_t$ are values from two different time series and we measure the similarity by calculating the ratio of corresponding values that match. They recommend using one quarter of the standard deviation of the normalized time series for ϵ . While we could do this as well, we think a user should set the value of ϵ . This makes a lot of sense for numeric time series, as there may be requirements from the application side concerning an acceptable *margin of error* of predictions, i.e., which distance of a prediction to the actual value can still be tolerated. Essentially, our approach determines the intrinsic predictability of a time series for a given margin of error and allows a user to investigate the tradeoffs between predictability and different margins of error.

B. Upper Bound for Predictability

First, we look at the uncertainty introduced by predicting time series values. More precisely, given the prediction $\hat{X}_t$ for timestamp t , how much information do we need to determine the actual value X_t ? In fact, we also have to figure out if $\hat{X}_t$ is a correct value or if we have made a prediction error. More formally, we have to determine the conditional entropy $H(E_t, X_t | \hat{X}_t)$, which is directly related to the entropy rate $\mathcal{H}(X)$. Since the prediction model uses the past observations to make a prediction, i.e., $\hat{X}_t = f(X_1, X_2, \dots, X_{t-1})$, $H(X_t | \hat{X}_t) = H(X_t | X_{t-1}, \dots, X_1)$. Applying Corollary 1 of the chain rule to $H(E_t, X_t | \hat{X}_t)$, we obtain

$$H(E_t, X_t | \hat{X}_t) = H(X_t | \hat{X}_t) + H(E_t | X_t, \hat{X}_t) \quad (11)$$

However, the entropy $H(E_t | X_t, \hat{X}_t)$ is equal to zero, since knowing X_t and $\hat{X}_t$, we can directly deduce E_t without any additional information, so

$$H(E_t, X_t | \hat{X}_t) = H(X_t | \hat{X}_t) = H(X_t | X_{t-1}, \dots, X_1) \quad (12)$$

Computing the average entropy $H(X | \hat{X})$ of predicting the whole time series then boils down to

$$H(X | \hat{X}) = \frac{1}{n} \sum_{t=1}^n H(X_t | X_{t-1}, \dots, X_1) \quad (13)$$

For sufficiently large values of n , this expression can be used to estimate the entropy rate (cf. Definition 4).

$$H(E_t, X_t | \hat{X}_t) = H(X | \hat{X}) \approx \mathcal{H}(X) \quad (14)$$

We now turn to determining the conditional entropy $H(E_t, X_t | \hat{X}_t)$. Applying the chain rule differently, we get

$$\mathcal{H}(X) \approx H(E_t, X_t | \hat{X}_t) = H(E_t | \hat{X}_t) + H(X_t | \hat{X}_t, E_t) \quad (15)$$

Intuitively, $H(E_t | \hat{X}_t)$ represents the information needed to communicate that an error was made, while $H(X_t | \hat{X}_t, E_t)$ stands for the information needed to determine X_t given the prediction $\hat{X}_t$ and the knowledge that an error occurred at time t . In its current form, Equation (15) is difficult, if not impossible, to compute. We would need access to a (large) number of joint and conditional probabilities and, more importantly, the random variables X_t and $\hat{X}_t$ are continuous. Nevertheless, we can provide an upper bound, using Definition 6 to handle continuous domains.

First, we look at $H(E_t | \hat{X}_t)$. Since conditionality reduces entropy (Equation (6)), we know that

$$\begin{aligned} H(E_t | \hat{X}_t) &\leq H(E_t) \\ &= -p(E_t = 0) \log_2 p(E_t = 0) \\ &\quad -p(E_t = 1) \log_2 p(E_t = 1) \end{aligned} \quad (16)$$

E_t is a discrete random variable, so we are able to apply Equation (1). We distinguish two cases for the right-most part of Equation (15):

$$\begin{aligned} H(X_t | \hat{X}_t, E_t) &= p(E_t = 0) H(X_t | \hat{X}_t, E_t = 0) + \\ &\quad p(E_t = 1) H(X_t | \hat{X}_t, E_t = 1) \end{aligned} \quad (17)$$

At this point, Song et al. [?] utilize the original Fano's inequality [?] (for the discrete case with exactly one correct value) and the first part for $p(E_t = 0)$ drops out, because making a correct prediction means that they do not need any additional information to determine the correct value (the prediction and the correct value are equal). This is not the case for our Equation (17), as $X_t \approx_\epsilon \hat{X}_t$, but not necessarily $X_t = \hat{X}_t$.

What we are computing is a resolution-dependent entropy rate. For $p(E_t = 0) H(X_t | \hat{X}_t, E_t = 0)$, we know that our prediction is considered correct. Thus, $X_t \approx_\epsilon \hat{X}_t$, which, according to Definition 6, means that the actual value x_t has to lie in $B_\epsilon(\hat{x}_t)$. We divide the domain into sub-intervals of length ϵ , starting from $\hat{x}_t$ (see Figure 3): $B_\epsilon(\hat{x}_t)$ is shaded in gray in the illustration. For $p(E_t = 1) H(X_t | \hat{X}_t, E_t = 1)$, we know that we made a prediction error and, consequently, $x_t \notin B_\epsilon(\hat{x}_t)$.

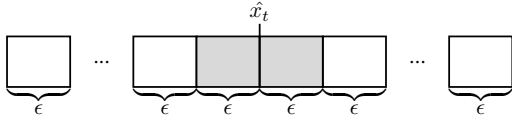


Fig. 3: Neighborhood around prediction

At resolution ϵ , we cannot distinguish all the values falling into $B_\epsilon(\hat{x}_t)$, we can only distinguish two cases: the correct value lies in the ϵ -cell to the left of $\hat{x}_t$ or in the ϵ -cell to the right of $\hat{x}_t$. In order to determine the options for an incorrect prediction, we need to estimate the total number of available ϵ -cells. Assuming a uniform distribution of the values in the domain, there are $N_{eps} = \lceil \frac{x_{\max} - x_{\min}}{\epsilon} \rceil$ ϵ -cells altogether. Outside of $B_\epsilon(\hat{x}_t)$ is the correct one.

We know that $\mathcal{H}(X) \leq H(E_t) + H(X_t | \hat{X}_t, E_t)$ (cf. Formulas (15), (16), and (17)). As we are trying to infer a bound on predictability based on entropy, we want a bound that is valid for all distributions. This means, if we choose the maximum possible entropy for a given probability $p(E_t = 0)$ and the inequality does not hold for a measured entropy rate $\mathcal{H}(X)$, then we know that this probability is impossible. As we are dealing with entropies, these become maximal if we assume a uniform distribution. For values outside of $B_\epsilon(\hat{x}_t)$, we set their probabilities to $\frac{p(E_t=1)}{N-2}$, for those in $B_\epsilon(\hat{x}_t)$ to $\frac{p(E_t=0)}{2}$. Combining Equations (15), (16), and (17) (and assuming a uniform distribution), we obtain

$$\begin{aligned} \mathcal{H}(X) &\leq -2 \frac{p(E_t=0)}{2} \log_2 \frac{p(E_t=0)}{2} \\ &\quad - (N-2) \frac{p(E_t=1)}{N-2} \log_2 \frac{p(E_t=1)}{N-2} \\ &= -p(E_t=0)(\log_2 p(E_t=0) - \log_2 2) \\ &\quad - p(E_t=1)(\log_2 p(E_t=1) - \log_2 (N-2)) \end{aligned} \quad (18)$$

We equate the probabilities $p(E_t = 0)$ and $p(E_t = 1)$ to predictability as stated in Equations (8), (9), and (10), so we switch to this notation (we also replace $\log_2 2$ with 1:

$$\begin{aligned} \mathcal{H}(X) &\leq -\Pi^{\hat{X}}(\log_2 \Pi^{\hat{X}} - 1) \\ &\quad - (1 - \Pi^{\hat{X}})(\log_2(1 - \Pi^{\hat{X}}) - \log_2(N-2)) \end{aligned} \quad (19)$$

We have now connected the entropy rate of a time series to predictability, where $\Pi^{\hat{X}}$ is the predictability of a specific prediction algorithm, the output of which is defined by the stochastic process $\hat{X}$. We are interested in the following: given these constraints, what is the predictability of the best theoretically possible prediction algorithm? Even though we do not know this algorithm, we can determine an upper bound $\Pi^{\max}$ of its predictability. Essentially, we want an upper bound for $\Pi^{\hat{X}}$, given the constraint $\mathcal{H}(X) \leq f(\Pi^{\hat{X}})$.

If f were a decreasing function, then $\Pi^{\hat{X}} \leq f^{-1}(\mathcal{H}(X))$, which would allow us to upper-bound $\Pi^{\hat{X}}$. While f is not strictly monotonic, we show that in the range that is relevant for us, it is decreasing. The derivative of f with respect to $\Pi^{\hat{X}}$ is equal to $f'(\Pi^{\hat{X}}) = \log_2 \left(\frac{2(1-\Pi^{\hat{X}})}{(N-2)\Pi^{\hat{X}}} \right)$, and is negative for $\Pi^{\hat{X}} \in (\frac{2}{N}, 1]$. Any algorithm with a predictability lower than

$\frac{2}{N}$ is not interesting to us, as a predictability of exactly $\frac{2}{N}$ models an approach that takes values completely at random. Any algorithm performing worse than this surely cannot be the best possible algorithm. Thus, we can use Formula (19) to find the upper bound $\Pi^{\max}$.

C. Effective Alphabet Size

We exploit a known relationship between the *expected recurrence time* (*ert*), i.e., when we expect to see a symbol again, and the alphabet size [?]. For the symbolic case, *ert_i* is equal to $\frac{1}{p(x_i)}$ for symbol x_i . Let N be the alphabet size,

$$N = \sum_{i=1}^N (p(x_i) \cdot \text{ert}_i) = \sum_{i=1}^N \left(p(x_i) \frac{1}{p(x_i)} \right) \quad (20)$$

For a given time series of length n , this becomes:

$$N = \sum_{i=1}^N \left(\frac{f_i}{n} \text{ert}_i \right) = \frac{1}{n} \sum_{i=1}^N (f_i \cdot \text{ert}_i) \quad (21)$$

where f_i is the number of occurrences of symbol x_i in a time series.

Switching to a concrete time series means that we can also calculate N by looking at the individual positions of elements and using the exact distances. Assume a time series of length n containing N different symbols, the frequency f_i of symbol i , and the positions $t_{i,1}, t_{i,2}, \dots, t_{i,f_i}$ in which it occurs. The distance $d_{i,j}$ between a symbol at position $t_{i,j}$ and its most recent previous appearance $t_{i,j-1}$ is equal to

$$d_{i,j} = \begin{cases} t_{i,j} - t_{i,j-1} & t_j \neq 1 \\ t_{i,1} + (n - t_{i,f_i}) & t_j = 1 \end{cases} \quad (22)$$

We treat the time series as circular, wrapping around from the beginning to the end. This ensures that we compute the same value as before and also covers the corner case of a symbol only occurring once. Additionally, wrapping around means that we cover the whole length of the time series:

$$\sum_{i=j}^{f_i} d_{i,j} = n \Leftrightarrow \sum_{i=1}^N \sum_{j=1}^{f_i} d_{i,j} = \sum_{i=1}^N n = n \cdot N \quad (23)$$

In order to compute these sums, we do not actually need to know N and f_i explicitly, we can just add the distances d_{t_k} for every individual position t_k , since every t_k has exactly one $d_{i,j}$ associated with it:

$$\frac{1}{n} \sum_{k=1}^n d_{t_k} = N \quad (24)$$

Example 1: The table below shows an example for a time series of length eight (wrapping around):

t	1	2	3	4	5	6	7	8
x_t	b	c	b	a	d	b	c	d
d_t	3	3	2	8	5	3	5	3

Summing up the values according to Equation (24) gives us 32 and dividing by eight results in four, which is the number of different symbols in this sequence. $\diamond$

The advantage of the reformulation in Equation (24) is that it can also be applied to numeric data. We just have to redefine d_{t_k} using tolerance threshold matching

$$d_{t_k} = \begin{cases} k - (\max_{x_{t_i} \approx_\epsilon x_{t_k}, i < k} i) & \text{if such an } i \text{ exists} \\ t_k + (n - \max_{x_{t_i} \approx_\epsilon x_{t_k}, i \geq k} i) & \text{otherwise} \end{cases} \quad (25)$$

Example 2: Let us now show an example with numeric values using Equation (25), assuming a threshold of $\epsilon = 1$:

t	1	2	3	4	5	6	7	8
x_t	1.2	3.5	5.3	4.1	2.6	6.2	1.9	5.0
d_t	2	5	3	2	3	3	2	5

Summing up and dividing gives us $25/8 = 3.125$. $\diamond$

There is one issue with applying Equation (24) in a straightforward manner to numeric data. The effective alphabet size we obtain is sensitive to the order in which the values appear. For instance, if the values in Example 2 appear in ascending order, from 1.2 up to 6.2, we would get an alphabet size of $14/8 = 1.75$. The order 1.2, 2.6, 4.1, 5.3, 1.9, 3.5, 5.0, 6.2 results in a size of $30/8 = 3.75$.

Averaging over all permutations is infeasible, but generating R (independent) random permutations of a given time series T gives us a sample consisting of the time series $T_1, T_2, \dots, T_R$. compute the effective alphabet size $N_1, N_2, \dots, N_R$ and the mean value

$$\bar{N}_R = \frac{1}{R} \sum_{r=1}^R N_r \quad (26)$$

The associated *standard error* on this mean, $\sigma_{\bar{N}_R}$, is given by

$$\sigma_{\bar{N}_R} = \frac{\sigma}{\sqrt{R}} \quad (27)$$

where σ is the *standard deviation* of the whole population, i.e., all permutations. As this is not known, we use the *sample standard deviation*, σ_N , instead

$$\sigma_{\bar{N}_R} \approx \frac{\sigma_N}{\sqrt{R}} \quad (28)$$

We now calculate confidence intervals for the true mean of the whole population (assuming a normal distribution). Striving for a 95% confidence interval means that we need to cover 95% of the area under the normal curve, which is the case within (approximately) 1.96 standard deviations of the mean. Thus, the 95% limits are defined by $[\bar{N}_R - 1.96\sigma_{\bar{N}_R}, \bar{N}_R + \sigma_{\bar{N}_R}]$. We can now determine how many samples we need to obtain an absolute error smaller than e :

$$\frac{1.96\sigma_N}{\sqrt{R}} \leq e \Rightarrow R \geq \left(\frac{1.96\sigma_N}{e} \right)^2 \quad (29)$$

For other levels of confidence, we have to replace 1.96 with the appropriate value.

VI. ENTROPY RATE ESTIMATION

Equation (19) makes it clear that being able to accurately estimate the entropy rate is crucial for our approach. As already indicated, there are numerous different ways to estimate the entropy rate of a time series. Many of these methods converge

to the true entropy rate when the length n of the time series goes to infinity. In this case, the differences between those estimators do not play a role, as their limit is always the true entropy rate. However, when applying different entropy rate estimators to finite time series, the differences do matter. Surprisingly, work on entropy rate estimation for finite time series is few and far between. The most detailed (empirical) study we found was by Lesne et al. [?].

A. Lempel-Ziv Encoding

Practical approaches based on Lempel-Ziv use the compressibility of a time series to estimate the entropy rate. This is done by parsing the time series and partitioning it into a set of distinct words. For the computation of the entropy rate, we are particularly interested in the number of words created by the parsing: the higher/lower the number of partitions, the higher/lower the entropy.

In principle, two different ways to parse a sequence are distinguished. The first one, which we call LZ77, processes a sequence by looking at previously parsed characters [?]. The second one, which we call LZ78, explicitly constructs a dictionary storing previously unseen subsequences encountered during the parsing [?]. We explain their mode of operation with an example. Assume we are given the following sequence: *abbaabaaabbababbbabaa*. LZ77 partitions the sequence in the following way: *a|b|ba|aba|aabb|abab|bbabaa*. LZ78, on the other hand, creates the following parsing: *a|b|ba|ab|aa|abb|aba|bb|bab|aa*.

Lesne et al. have shown that LZ78 has a tendency to overestimate the true entropy rate, especially for low-entropy scenarios [?]. A low level of entropy means that we expect a lot of long patterns repeating in a sequence. However, as LZ78 has to gradually fill the dictionary with all the shorter previously unseen patterns, it takes a while until longer patterns appear in the dictionary. For example, the sequence *abababababababab* contains many long patterns that repeat. However, when creating the dictionary, LZ78 has to go through a lot of shorter patterns resulting in the following parsing: *a|b|ab|aba|ba|bab|abab*. In theory, this effect does not matter, as we compute the limit for the length of the sequence going to infinity. When applying the estimation to finite sequences, the impact is noticeable [?]. The estimation of LZ77 is much more stable, i.e., there is no discernible effect of the entropy level on the estimation. In general, this variant tends to underestimate the true value, though.

For our purposes, this makes LZ77 a much better choice. As we determine an upper bound for the predictability, which is inversely related to the entropy rate, underestimating the entropy rate will not put us into danger of exceeding this upper bound. In earlier experiments, we actually observed that entropy estimations made with LZ78 led to situations in which we obtained a value for $\Pi^{\max}$ (using Equation (19)) that was not an upper bound [?]. On the flip side, using LZ77 we lose some tightness.

Kontoyiannis et al. have improved LZ77 further by averaging out fluctuations [?]. Instead of shifting the parsing by the length of the most recently parsed word and starting to

match the next word from there, Kontoyiannis et al. match words from *every* position in the sequence. We give a formal definition in the following.

Definition 8: Given a time series $T = (x_1, x_2, \dots, x_n)$, we define λ_t to be the length of the phrase starting at position t as encoded by LZ77, i.e., this is the shortest (novel) prefix of the time series starting at time point t that does not appear as a contiguous sequence in $x_1, x_2, \dots, x_{t-1}$. Let Λ denote the vector $(\lambda_1, \lambda_2, \dots, \lambda_n)$.

Towards the end of a sequence, we usually run out of novel prefixes, when all remaining subsequences have already been seen before. Kontoyiannis et al. do not explicitly state how this case is handled, as they are mainly interested in the asymptotic behavior, which means that sequences are (potentially) infinite. However, they provide an alternative definition for the shortest novel prefix: take the longest matching prefix and add one to it. This rule, which we call *lmp+*, can also be applied to non-novel prefixes. In addition to this, there are various implementations of the approach by Kontoyiannis, such as the one by Smart et al. [?], that slightly modify the approach, mainly for performance reasons: if they are not able to find a novel prefix for position t , they set λ_t to 0. We call this method *0-fill*.

Example 3: Given the sequence *abbaabaaabba*, we now compute the values for λ_t . The entries for the positions 9 to 12 show the values for the alternatives *lmp+* and *0-fill*, respectively, as described above.

t	1	2	3	4	5	6	7	8	9	10	11	12
λ_t	1	1	2	2	3	4	3	4	5/0	4/0	3/0	2/0

◇

Once all the values for Λ are determined, Kontoyiannis et al. use the average length of the novel patterns to estimate the entropy rate.

Definition 9: The entropy rate estimate $\mathcal{H}_{est}$ as defined by Kontoyiannis et al. [?] is equal to

$$\mathcal{H}_{est} = \frac{\log_2(n)}{\frac{1}{n} \sum_{t=1}^n \lambda_t} \quad (30)$$

The primary challenge in estimating the entropy rate lies in computing Λ efficiently, which we turn to next.

B. Efficient Computation

Since we cannot apply fast algorithms based on suffix trees, due to our tolerance threshold matching (see Section IV-B), we have to fall back on other techniques. Clearly, we want to avoid using a naive approach that, given a sequence of length n , traverses it position by position and computes the longest common contiguous subsequence between a prefix of the current position and the already parsed part of the sequence in a brute-force manner. This would result in an approach with a complexity of $O(n^3)$. Instead we draw on different results for computing longest common subsequences and substrings utilizing dynamic programming [?], [?], [?].

Given a time series $T = (x_1, x_2, \dots, x_n)$ containing numeric values, one of our goals is to create an $n \times n$ matrix M ,

in which the entry $M[t, t']$ tells us the length l of the longest matching contiguous sequence of a prefix starting at t and a subsequence starting at t' , with $t' + l - 1 < t$. Essentially, using a tolerance threshold of ϵ , $M[t, t'] = l$ means that $x_t \approx_\epsilon x_{t'}$, $x_{t+1} \approx_\epsilon x_{t'+1}$, $\dots$, $x_{t+l-1} \approx_\epsilon x_{t'+l-1}$.

Example 4: Assume the numeric time series $T = (1.2, 3.5, 4.1, 2.1, 1.8, 3.8)$ and a tolerance threshold of $\epsilon = 1$. Then M contains the following entries (the positions with an asterisk are empty, since for these $t' < t$ does not hold):

$t \setminus t'$	1	2	3	4	5	6
1	*	*	*	*	*	*
2	0	*	*	*	*	*
3	0	2	*	*	*	*
4	1	1	1	*	*	*
5	2	0	0	0	*	*
6	0	1	1	0	0	*

For example, $M[5, 1]$ is equal to 2, because $x_5 \approx_\epsilon x_1$ and $x_6 \approx_\epsilon x_2$. ◇

If we had a matrix like this, we could easily compute Λ : for both, *lmp+* and *0-fill*, $\lambda_t = \max_{1 \leq t' < t} M[t, t'] + 1$, with the additional constraint for *0-fill* that $\lambda_t = 0$ if $t + \max_{1 \leq t' < t} M[t, t'] > n$.

Building the matrix top-down (starting with the first row) is suboptimal: every time we found a match, we would have to update the lengths of the sequences along a diagonal above. In order to avoid this, we build the matrix bottom-up, starting with the last row ($t = n$), which is initialized in the following way: for $t' < t$, $M[t, t'] = 1$ if $x_t \approx_\epsilon x_{t'}$, otherwise $M[t, t'] = 0$. The rest of the matrix is filled using the recurrence relation

$$M[t, t'] = \begin{cases} \min(M[t+1, t'+1] + 1, t - t') & \text{if } x_t \approx_\epsilon x_{t'} \\ 0 & \text{otherwise} \end{cases} \quad (31)$$

This formula can be directly derived from the one used for minimum-editing distances (dropping the expressions for insertions and deletions), described e.g. in [?]. Comparing it to $t - t'$ and taking the minimum makes sure that the matching subsequence starting at t' does not extend beyond $t - 1$.

What is still missing is the computation of the matching pairs (t, t') for which $x_t \approx_\epsilon x_{t'}$. As we need to fill the matrix bottom-up, we compute the pairs using a reverse nested loop, which is easy to do, because the time series is sorted by timestamp. Also, we do not need to keep the whole matrix in memory, we only require the current row and the row created in the previous step. Algorithm 1 shows the pseudocode for a baseline algorithm computing Λ in quadratic time with a linear space complexity.

VII. OUR SOLUTION

The entropy rate estimator introduced in Section VI requires computing the vector $\Lambda = (\lambda_1, \lambda_2, \dots, \lambda_n)$ of Lempel-Ziv coefficients for every position of the time series (Definition 8). As established in Section IV-B, the non-transitivity of $\approx_\epsilon$ prevents the use of suffix trees or suffix arrays, and a naive brute-force computation incurs $O(n^3)$ time. The baseline algorithm (Algorithm 1) reduces this to $O(n^2)$ via dynamic

Algorithm 1: Baseline algorithm

Input : Array x of length n , threshold ϵ , $\text{Imp+}/0\text{-fill}$
Output: $\Lambda = (\lambda_1, \lambda_2, \dots, \lambda_n)$

```

1 Initialize  $\Lambda$  with  $(1, 1, \dots, 1)$ ;
2 Initialize  $\text{curr}[1..n]$  with 0;
3 for  $t \leftarrow n$  to 2 do
4   for  $t' \leftarrow t - 1$  to 1 do
5     if  $x_t \approx_\epsilon x_{t'}$  then
6       if  $t < n$  then
7          $\text{curr}[t'] \leftarrow \min(\text{prev}[t' + 1] + 1, t - t')$ ;
8       else
9          $\text{curr}[t'] \leftarrow 1$ ; // last row in  $M$ 
10    if  $\text{Imp+}$  or ( $0\text{-fill}$  and  $\text{curr}[t'] \leq n$ ) then
11       $\lambda_t \leftarrow \max(\lambda_t, \text{curr}[t'] + 1)$ ;
12    else
13       $\lambda_t \leftarrow \infty$ ;
14  if  $\lambda_t = \infty$  then
15     $\lambda_t \leftarrow 0$ ; // 0-fill
16   $\text{prev} \leftarrow \text{curr}$ ;
17  set  $\text{curr}[1..t - 1]$  to 0;
18 return  $\Lambda$ 

```

programming, but it traverses *all* prior positions $t' < t$ for every position t in a reversed nested loop—regardless of whether $x_{t'}$ is within ϵ of x_t . When the time series takes values over a large range relative to ϵ , most pairs (t, t') fail the ϵ -matching condition, and the full nested-loop traversal wastes the majority of its work.

We propose an efficient algorithm, the *Inverted File Index* (IFI), that restricts the candidate set for each position t to those prior positions $t' < t$ whose values lie within the ϵ -range of x_t . An index structure locates this candidate set in time sublinear in n , avoiding redundant comparisons that inflate the constant factor of the baseline.

A. Problem Formulation as a Self-Join

We first observe that identifying all ϵ -matching prior positions for a given t is equivalent to a self-join with inequality predicates. We represent the time series $T = (x_1, x_2, \dots, x_n)$ as a relational table r with attributes (t, ℓ, h) , where $\ell = x_t - \epsilon$ and $h = x_t + \epsilon$ are the lower and upper bounds of the ϵ -interval around each value. The condition $x_t \approx_\epsilon x_{t'}$ (Definition 6) is equivalent to the two intervals $[\ell, h]$ and $[\ell', h']$ overlapping, i.e., $\ell \leq h'$ and $h \geq \ell'$. Adding the temporal ordering $t' < t$ yields the self-join:

$$Q_0: r \bowtie r' \text{ s.t. } r.t > r'.t \wedge r.\ell \leq r'.h \wedge r.h \geq r'.\ell. \quad (32)$$

The output of Q_0 is the set of pairs (t, t') such that $x_t \approx_\epsilon x_{t'}$ and $t' < t$ —precisely the non-zero entries in the first row of the dynamic programming matrix M introduced in Section VI. Computing Λ then amounts to filling the remaining rows of M by propagating Q_0 's output along anti-diagonals according to the recurrence (31).

B. Inverted File Index

The two interval-overlap conditions in Q_0 simplify to a single range constraint on values: for the current position t , all matching prior positions $t' < t$ have $x_{t'} \in [x_t - 2\epsilon, x_t + 2\epsilon]$. (The factor of 2ϵ arises because the intervals $[x_t - \epsilon, x_t + \epsilon]$ and $[x_{t'} - \epsilon, x_{t'} + \epsilon]$ overlap if and only if $|x_t - x_{t'}| \leq 2\epsilon$.) This range structure is exploited by an *inverted file index* over the distinct values in the time series.

Definition 10: The **Inverted File Index** (IFI) for a time series $T = (x_1, x_2, \dots, x_n)$ is a dictionary $\mathcal{D}$ mapping each distinct value v appearing in T to the sorted list of positions where v occurs. A sorted array K of all distinct keys of $\mathcal{D}$ serves as a directory for range queries.

Building the IFI requires one pass over T and costs $O(n + N_v \log N_v)$ time, where N_v is the number of distinct values (the $\log N_v$ factor accounts for sorting K), and $O(n)$ space. Given the index, all prior positions $t' < t$ satisfying $x_t \approx_\epsilon x_{t'}$ are found by binary search on K for the leftmost key in $[x_t - 2\epsilon, x_t + 2\epsilon]$, followed by a forward scan through matching keys collecting the associated position lists (retaining only positions $t' < t$).

Algorithm 2: IFI_BUILDIDX: Construct the Inverted File Index

Input : Time series $x[1..n]$, length n
Output: Dictionary $\mathcal{D}$, sorted key array K

```

1  $\mathcal{D} \leftarrow \emptyset$ ;
2 for  $i \leftarrow 1$  to  $n$  do
3   if  $x[i] \notin \mathcal{D}$  then
4      $\mathcal{D}[x[i]] \leftarrow \text{empty list}$ ;
5     append  $i$  to  $\mathcal{D}[x[i]]$ ;
6  $K \leftarrow \text{array of keys of } \mathcal{D}, \text{ sorted ascending}$ ;
7 return  $\mathcal{D}, K$ ;

```

Algorithm 3: IFI_LOOKUP: Retrieve ϵ -matching prior positions

Input : Sorted key array K , dictionary $\mathcal{D}$, query value s , current position t , threshold ϵ
Output: List of matching prior positions

```

1  $\text{matches} \leftarrow \text{empty list}$ ;
2  $lb \leftarrow \text{first index in } K \text{ with } K[lb] \geq s - 2\epsilon$ ; // binary search
3  $rb \leftarrow \text{first index in } K \text{ with } K[rb] > s + 2\epsilon$ ; // binary search
4 for  $k \leftarrow lb$  to  $rb - 1$  do
5   for  $t' \in \mathcal{D}[K[k]]$  do
6     if  $t' \geq t$  then break;
7     append  $t'$  to  $\text{matches}$ ;
8 return  $\text{matches}$ ;

```

C. Dynamic Programming with the IFI

Given the candidate prior positions for x_t returned by IFI_LOOKUP, we compute λ_t using dynamic programming on

M . The recurrence (31) requires only two consecutive rows of M : for position t (the current row) and for position $t + 1$ (the next row, already computed). We therefore maintain two sparse dictionaries, *curr_row* and *next_row*, indexed by the prior position t' : a stored value *curr_row* $[t'] = k$ records that $M[t, t'] = k$, i.e., the longest ϵ -matching prefix beginning at (t, t') has length k .

The recurrence translates to:

$$\text{curr_row}[t'] = \min(\text{next_row}[t'+1]+1, t-t') \quad \text{if } x_t \approx_\epsilon x_{t'}, \quad (33)$$

where *next_row* $[t' + 1]$ carries the diagonal value from the preceding row, and $t - t'$ caps the match length to prevent the prior substring from overlapping the current position. After processing all matching t' , the coefficient λ_t is updated as:

$$\lambda_t = \max(\lambda_t, \text{curr_row}[t'] + 1) \quad \text{for each matched } t'. \quad (34)$$

A flag array *lzc_zeros* $[1..n]$ tracks positions where no match exists at the boundary, enabling the 0-fill convention: after the main loop, λ_t is set to 0 for flagged positions. If no match exists for x_t at all (*matches* = $\emptyset$), the diagonal chain cannot continue and both dictionaries are reset; in this case $\lambda_t = 1$ by definition.

Algorithm 4: IFI: Compute Λ via the Inverted File Index

Input : Array $x[1..n]$, threshold ϵ , lmp+/0-fill convention

Output: $\Lambda = (\lambda_1, \lambda_2, \dots, \lambda_n)$

```

1  $\mathcal{D}, K \leftarrow \text{IFI\_BUILDIDX}(x, n)$ ;
2  $\Lambda[1..n] \leftarrow 1$ ;
3 lzc_zeros $[1..n] \leftarrow 0$ ;
4 curr_row  $\leftarrow \emptyset$ ; next_row  $\leftarrow \emptyset$ ;
5 for  $t \leftarrow n$  to 1 do
6   matches  $\leftarrow \text{IFI\_LOOKUP}(K, \mathcal{D}, x[t], t, \epsilon)$ ;
7   if matches  $\neq \emptyset$  then
8     next_row  $\leftarrow \text{curr\_row}$ ; curr_row  $\leftarrow \emptyset$ ;
9     for  $t' \in \text{matches}$  do
10      diag  $\leftarrow 1 + \text{next\_row.get}(t' + 1, 0)$ ;
11      curr_row $[t'] \leftarrow \min(\text{diag}, t - t')$ ;
12       $\lambda' \leftarrow \text{curr\_row}[t'] + 1$ ;
13      if lmp+ or  $t + \lambda' - 1 \leq n$  then
14         $\Lambda[t] \leftarrow \max(\Lambda[t], \lambda')$ ;
15      else
16        lzc_zeros $[t] \leftarrow 1$ ;
17   else
18     curr_row  $\leftarrow \emptyset$ ; next_row  $\leftarrow \emptyset$ ;
19 for  $t \leftarrow 1$  to  $n$  do
20   if lzc_zeros $[t] = 1$  then  $\Lambda[t] \leftarrow 0$ ;
21 return  $\Lambda$ ;
```

D. Complexity Analysis

Table I summarizes the complexity of all algorithms considered. *Brute Force* checks every pair (t, t') and every candidate

TABLE I: Time and space complexity of algorithms for computing Λ under ϵ -matching. K is the number of ϵ -matching pairs (t, t') output by Q_0 ; N_v is the number of distinct values.

Algorithm	Time	Space
Brute Force (BF)	$O(n^3)$	$O(1)$
RNLJ (baseline)	$\Theta(n^2)$	$O(n)$
IFI (proposed)	$O(n^2)$	$O(n)$
Forest of Interval Trees (FIT)	$O(n^3 \log n)$	$O(n^3)$
Inequality Self-Join (IESJ)	$O(n^2 + K \log K)$	$O(n + K)$

length in three nested loops: $O(n^3)$ time and $O(1)$ space. *RNLJ* (Algorithm 1) processes all $n(n - 1)/2$ pairs in a reversed nested loop and propagates diagonal values; it spends $\Theta(n^2)$ time unconditionally regardless of how many pairs satisfy the ϵ -condition.

IFI (Algorithm 4): index construction costs $O(n + N_v \log N_v)$; for each of the n positions, *IFI_LOOKUP* costs $O(\log N_v)$ for the binary search plus time proportional to the number of actual matches. In the worst case (all values within 2ϵ of each other) *IFI* visits all $O(n^2/2)$ pairs, matching *RNLJ*; in practice, when the data range is large relative to ϵ , many fewer pairs are matched and *IFI* is substantially faster.

FIT maintains a growing dictionary of interval trees indexed by match length; each time step inserts $O(n)$ segments, giving $O(n^3 \log n)$ time and $O(n^3)$ space—impractical for large series. *IESJ* first materializes all K matching pairs (requiring $O(K)$ space, potentially $O(n^2)$) and then applies DP; sorting costs $O(K \log K)$, making *IESJ* more expensive than *IFI* when K is large.

IFI dominates the alternatives: it avoids the exhaustive storage of *FIT*, the upfront materialization cost of *IESJ*, and the unconditional per-pair evaluation of *RNLJ*, while maintaining $O(n)$ memory at all times.

E. Correctness and Design Choices

a) *Two-dictionary sufficiency.*: The recurrence $M[t, t'] = \min(M[t + 1, t' + 1] + 1, t - t')$ depends on the entry one step forward in both dimensions. By iterating t from n down to 1, the value $M[t + 1, t' + 1]$ is available as *next_row* $[t' + 1]$ from the preceding iteration. Positions t' for which $x_t \not\approx_\epsilon x_{t'}$ contribute $M[t, t'] = 0$ and are omitted from both dictionaries, keeping them sparse.

b) *Handling empty matches.*: When *matches* = $\emptyset$, no diagonal chain from row $t + 1$ can extend to row t . Resetting both dictionaries prevents stale values from propagating; $\lambda_t = 1$ is the base case by Definition 8.

c) *lmp+ vs. 0-fill.*: Both conventions are supported by the *lzc_zeros* flag. Under the lmp+ convention, a match reaching the boundary ($t + \lambda' > n$) is allowed and λ_t is set to the truncated length; under 0-fill, such positions are flagged and set to 0 after the main loop. As discussed in Section VI, lmp+ produces a slightly higher Λ sum and thus a slightly lower entropy estimate, which reinforces the upper-bound property of $\Pi^{\max}$. We recommend lmp+ for applications where ensuring the upper bound is paramount.

d) *Relationship to the published estimator.*: The published work [?] implemented NLZ2 using a sorted inverted index for ϵ -matching value lookup but computed each λ_t

independently, without propagating diagonal DP values across positions. This is equivalent to setting $next_row[t' + 1] = 0$ for all t' in every outer iteration of IFI. Maintaining $next_row$ across iterations—the key innovation of IFI—reuses diagonal chains established in prior iterations rather than recomputing them from scratch, reducing the effective constant factor of the $O(n^2)$ runtime.

(** sh: boundary **)

VIII. EXPERIMENTAL EVALUATION

We design five experiments addressing the principal claims of the paper. Experiments VIII-B and VIII-C assess the validity of $\Pi^{\max}$ as an upper bound and the accuracy of the entropy rate estimator. Experiment VIII-D evaluates the computational advantage of IFI over alternative algorithms. Experiment VIII-E investigates the sensitivity of $\Pi^{\max}$ to the tolerance parameter ϵ . Experiment VIII-F examines whether $\Pi^{\max}$ provides a model-independent characterization of the inherent predictability of a time series.

A. Experimental Setup

a) Synthetic datasets. We generate synthetic time series using first-order Markov chains with $N_s \in \{5, 10, 15, \dots, 50\}$ states and varying transition distributions, denoted $N_x D_y$: x gives the number of states and y specifies the distribution (u = uniform transitions; s = skewed Zipf with $\alpha = 2$; $y = i \in \{0, 1, \dots, 7\}$ for a parameterized family with $N_s - 1$ weak transitions each with probability $p_w = 1/(10iN_s)$ and one dominant transition taking the remaining probability). Numeric values are generated by mapping each state to a real value drawn once uniformly from $[0, 1]$, enabling tolerance-based comparisons. The exact entropy rate is computed from the Markov chain’s transition matrix and stationary distribution.

b) Real-world datasets. We use four real-world time series: (1) **ETTh1** (hourly electricity transformer temperatures over two years); (2) **S&P 500** (daily open prices, 1990–2021); (3) **Temperature** (hourly measurements, city of Zurich); (4) **Precipitation** (daily rainfall, CPC-CONUS, 1948–2024).

c) Prediction models. We train ARIMA (order 1), LSTM, CNN-LSTM, SciNet, and (for synthetic data) a first-order Markov predictor (FOMM) that exactly matches the generating process. All models are trained on the first 80% and evaluated on the remaining 20% of each series. Realized predictability Π^{model} is the fraction of predictions within ϵ of the true value (Eq. (10)).

d) Implementation. All algorithms are implemented in Python. The numerical solver `scipy.optimize.fsolve` is used to invert Eq. (19) and obtain $\Pi^{\max}$ from the estimated entropy rate. All experiments are run on [hardware to fill in] with [RAM to fill in] GB of memory; runtimes are wall-clock times averaged over five independent runs.

B. Experiment 1: Validity of $\Pi^{\max}$ as an Upper Bound

Research question. Is $\Pi^{\max}$, computed via the IFI Kontoyiannis estimator and Eq. (19), a valid upper bound on the predictability of any model?

Hypothesis. For stationary ergodic series (synthetic Markov chains), $\Pi^{\max} \geq \Pi^{\text{model}}$ for all models and all entropy levels. The FOMM predictor, which matches the generating process, should approach $\Pi^{\max}$ most closely.

Setup. We use 20 synthetic datasets $N_x D_y$ with $x \in \{5, 10, \dots, 50\}$ and $y \in \{u, s\}$, setting $\epsilon = 0$ (exact symbolic matching, so the numeric and symbolic frameworks coincide). We compare $\Pi^{\max}$ (IFI and RNLJ estimators) with Π^{FOMM} and $\Pi^{\text{AR}(1)}$, as well as the exact Π computed from the Markov chain.

Metrics. (i) Whether $\Pi^{\max} \geq \Pi^{\text{model}}$ for every dataset (binary; any violation falsifies the bound); (ii) gap $\Pi^{\max} - \Pi^{\text{FOMM}}$ (tightness).

Falsification. A single dataset where $\Pi^{\text{model}} > \Pi^{\max}$ falsifies the upper bound claim unconditionally.

Visualization. Two line plots (one per distribution family u, s): x-axis = number of states N_s ; y-axis = predictability. Curves: $\Pi^{\max}$ (IFI), $\Pi^{\max}$ (RNLJ), Π^{FOMM} , $\Pi^{\text{AR}(1)}$, Π^{exact} . The IFI and RNLJ curves for $\Pi^{\max}$ must lie above all model curves.

C. Experiment 2: Accuracy of Entropy Rate Estimation

Research question. How accurately does the IFI-based Kontoyiannis estimator approximate the true entropy rate, and how does accuracy depend on the entropy level?

Hypothesis. IFI and RNLJ implement the same Kontoyiannis formula and should produce identical entropy estimates (any discrepancy would indicate an implementation error). For moderate-entropy series the estimator converges toward the exact rate; for low-entropy series it may overestimate (risking the upper bound); for high-entropy series it underestimates (a conservative but safer regime).

Setup. We use two synthetic families: (a) $N_{10} D_i$ with $i \in \{0, 1, \dots, 7\}$ (increasing skew, decreasing entropy); (b) $N_x D_u$ with $x \in \{2, 3, \dots, 50\}$ (increasing states, increasing entropy). We compare $\mathcal{H}_{\text{IFI}}$, $\mathcal{H}_{\text{RNLJ}}$, and $\mathcal{H}_{\text{exact}}$.

Metrics. Absolute error $|\mathcal{H}_{\text{est}} - \mathcal{H}_{\text{exact}}|$, relative error, sign of the error (over- vs. underestimation).

Falsification. Systematic overestimation across the full tested range would cause $\Pi^{\max}$ to fail as an upper bound (Experiment 1 would detect this; Experiment 2 identifies the root cause).

Visualization. Two plots: (a) estimated and exact entropy rate vs. skew index i (linear and log scale); (b) estimated and exact entropy rate vs. number of states N_s (linear scale). Both plots should show the estimator tracking the true rate, with deviation annotated at extremes.

D. Experiment 3: Runtime and Scalability of IFI

Research question. Does IFI achieve practical speedups over RNLJ and the other algorithms? How does runtime scale with series length n and tolerance ϵ ?

Hypothesis. IFI and RNLJ both have $O(n^2)$ worst-case runtime, but IFI is faster in practice for typical ϵ values (where only a fraction of pairs satisfy ϵ -matching). FIT should be the slowest ($O(n^3 \log n)$). IESJ may be competitive but will incur higher memory than IFI.

Setup. We vary $n \in \{500, 1000, 2000, 5000, 10000, 20000\}$ and $\epsilon \in \{\epsilon_{\min}, \epsilon_{25\%}, \epsilon_{50\%}, \epsilon_{75\%}, \epsilon_{\max}\}$, where $\epsilon_{\min}$ imposes exact matching, $\epsilon_{p\%}$ is chosen so that $p\%$ of all pairs satisfy ϵ -matching, and $\epsilon_{\max}$ makes all pairs match. We test on synthetic ($N_{50}D_u$) and real-world (Temperature) series. Algorithms compared: BF, RNLJ, IFI, IESJ. (FIT is included only for small n due to its cubic space requirement.)

Metrics. Wall-clock time (seconds), peak resident memory (MB), speedup ratio IFI/RNLJ.

Falsification. If IFI is not faster than RNLJ for $\epsilon \leq \epsilon_{50\%}$, the practical motivation for IFI is weakened (the $O(n)$ memory advantage remains, but the practical argument is reduced).

Visualization. (a) Log-log plot of runtime vs. n for each algorithm at $\epsilon_{25\%}$: slopes should be consistent with theoretical complexities (slope ≈ 2 for IFI and RNLJ, slope ≈ 3 for BF); (b) line plot of runtime vs. ϵ (as fraction of pairs matched) for IFI and RNLJ at fixed n , showing IFI speedup shrinking as ϵ increases; (c) bar chart of peak memory for each algorithm at the largest n .

E. Experiment 4: Sensitivity of $\Pi^{\max}$ to ϵ

Research question. How does $\Pi^{\max}$ vary with ϵ ? Does the bound remain valid and informative across the tested range?

Hypothesis. As ϵ increases, the effective alphabet size N decreases and the entropy rate decreases, so $\Pi^{\max}$ increases monotonically. Realized model performance Π^{model} also increases with ϵ but remains below $\Pi^{\max}$. The bound is tightest (most informative) at small ϵ and loosens as ϵ grows.

Setup. We sweep ϵ from $\epsilon_{\min}$ to $\epsilon_{\max}$ over all four real-world datasets (ETTh1, S&P 500, Temperature, Precipitation) and compute $\Pi^{\max}$ (IFI) alongside Π^{model} for ARIMA, LSTM, CNN-LSTM, and SciNet.

Metrics. $\Pi^{\max}$, Π^{model} for each model, gap $\Pi^{\max} - \max_{\text{model}} \Pi^{\text{model}}$ (tightness), and an upper-bound validity flag for every (dataset, ϵ) pair.

Falsification. Non-monotone $\Pi^{\max}$ with ϵ would indicate an error in the effective alphabet size computation. Any (dataset, ϵ) pair where $\Pi^{\text{model}} > \Pi^{\max}$ challenges the bound under non-stationary conditions and must be discussed.

Visualization. Four multi-panel figures (one per dataset): (a) predictability vs. ϵ (curves for $\Pi^{\max}$ and each model); (b) entropy rate vs. ϵ (monotone decreasing). Panels (a) should show $\Pi^{\max}$ lying above all model curves for all ϵ values.

F. Experiment 5: $\Pi^{\max}$ as a Model-Independent Characterization

Research question. Does $\Pi^{\max}$ provide a meaningful, model-independent characterization of the inherent predictability of a time series? Do datasets with higher $\Pi^{\max}$ consistently admit better model performance?

Hypothesis. Across diverse datasets, $\Pi^{\max}$ ranks datasets by predictability: a dataset with higher $\Pi^{\max}$ should show higher Π^{model} for all tested models. The gap $\Pi^{\max} - \Pi^{\text{model}}$ measures model suboptimality and varies across models but should be non-negative for all datasets.

Setup. We fix ϵ at a dataset-specific relative level (e.g., one quarter of the standard deviation of the normalized series,

following [?]) so that comparisons across datasets are on a common scale. We compute $\Pi^{\max}$ and Π^{model} for all real-world datasets and a representative set of synthetic datasets spanning a wide entropy range.

Ablation. We also compute $\Pi^{\max}$ using the discretization-based approach (applying Song et al.'s method after binning with bin width ϵ) and compare with the tolerance-based $\Pi^{\max}$, reproducing the motivating result from Section IV: the discretized version should fail to be an upper bound for at least some datasets or bin widths.

Metrics. Scatter plot of $\Pi^{\max}$ vs. Π^{model} across (dataset, model) pairs; Spearman rank correlation; gap $\Pi^{\max} - \Pi^{\text{model}}$; number of datasets for which the discretized bound fails.

Falsification. If $\Pi^{\max}$ does not correlate with achievable model performance across datasets, its utility as an intrinsic characterization is limited.

Visualization. (a) Scatter plot: one point per (dataset, model) pair; x-axis = $\Pi^{\max}$, y-axis = Π^{model} ; all points should lie below the diagonal $y = x$; (b) grouped bar chart of $\Pi^{\max}$ (tolerance-based vs. discretization-based) and Π^{model} across datasets, ordered by tolerance-based $\Pi^{\max}$.

ACKNOWLEDGMENTS

xxx

TABLE II: Mapping of principal claims to validating experiments and visualizations. All claims must be addressed before submission.

Claim	Experiment	Key Visualization
$\Pi^{\max}$ is a valid upper bound for the intrinsic predictability of any predictor on stationary numeric time series	Exp. 1 (§VIII-B)	Line plot: $\Pi^{\max}$ lies above all model curves for all synthetic datasets
Naive discretization (binning) causes $\Pi^{\max}$ to fail as an upper bound for numeric series	Exp. 5 ablation (§VIII-F)	Bar chart: discretized $\Pi^{\max}$ falls below at least one model curve
The Kontoyiannis estimator can accurately estimate the entropy rate of a numeric time series from finite observations	Exp. 2 (§VIII-C)	Line plots: estimated vs. exact entropy rate across entropy levels; error magnitude and sign annotated
IFI achieves the same $O(n^2)$ worst-case runtime as RNLJ with reduced practical cost via index-based pruning	Exp. 3 (§VIII-D)	Log-log runtime vs. n ; runtime vs. ϵ showing IFI advantage at low match rates
$\Pi^{\max}$ increases monotonically with ϵ and remains a valid upper bound on real-world datasets under appropriate estimator choice	Exp. 4 (§VIII-E)	Predictability vs. ϵ curves per dataset; $\Pi^{\max}$ above all model curves
$\Pi^{\max}$ is a model-independent characterization of predictability: higher $\Pi^{\max}$ implies higher achievable performance across models	Exp. 5 (§VIII-F)	Scatter plot: all (dataset, model) points below the $y = x$ diagonal; Spearman rank correlation